



VANLANG UNIVERSITY
School of Technology
Information Technology

CƠ SỞ DỮ LIỆU

(Databases)

CHƯƠNG 5: NGÔN NGỮ SQL



GVGD:

1. THS. LÝ THỊ HUYỀN CHÂU
2. THS. NGUYỄN THỊ QUYÊN
3. TS. VŨ THỊ MỸ HẰNG
4. THS. NGÔ QUỐC HUY
5. THS. HOÀNG CÔNG TRÌNH



HỌC KỲ III – NĂM HỌC 2023-2024



K29T



NỘI DUNG

- 1. Định nghĩa các thuật ngữ.**
- 2. Các ưu điểm của ngôn ngữ SQL.**
- 3. Định nghĩa CSDL dùng ngôn ngữ định nghĩa dữ liệu SQL.**
- 4. Viết các truy vấn (query) dùng SQL.**
- 5. Các loại phép kết.**
- 6. Các loại truy vấn con (subquery).**
- 7. Các khung nhìn (view)**



Giới thiệu SQL

- ***Ngôn ngữ truy vấn có cấu trúc*** (SQL - *Structured Query Language*) là một ngôn ngữ chuẩn được dùng để tạo lập và truy vấn các cơ sở dữ liệu quan hệ.
- SQL là một ngôn ngữ chuẩn cho các hệ quản trị CSDL quan hệ (RDBMS - *Relational DBMS*).



Các đặc điểm của ngôn ngữ SQL

- Ngôn ngữ SQL là một **ngôn ngữ tựa tiếng Anh** (*English-like language*), sử dụng các từ như *select, insert, delete* trong tập lệnh.
- Ngôn ngữ SQL là một **ngôn ngữ phi thủ tục** (*nonprocedural language*).
 - Chỉ ra các thông tin **gì** cần thiết (*what*).
 - **Không** cần phải chỉ ra cách thực hiện **như thế nào** (*how*) để có được các thông tin này.
- SQL xử lý các tập hợp mẫu tin (bảng) hơn là mỗi lần một mẫu tin đơn lẻ.



Các đặc điểm của ngôn ngữ SQL

- **SQL cung cấp nhiều lệnh cho nhiều công việc khác nhau:**
 - Truy vấn dữ liệu.
 - Thêm vào, cập nhật và xóa bỏ các hàng của bảng.
 - Tạo lập, thay đổi và xóa bỏ các đối tượng CSDL.
 - Điều khiển truy xuất cơ sở dữ liệu và các đối tượng CSDL.
 - Bảo đảm tính nhất quán của CSDL.



Ngôn ngữ SQL

- **Ngôn ngữ định nghĩa dữ liệu**
 - DDL - *Data Definition Language*
 - Các lệnh dùng để định nghĩa CSDL: tạo lập (create), thay đổi(alter) và hủy bỏ (drop) các đối tượng dữ liệu, thiết lập các ràng buộc.
- **Ngôn ngữ thao tác dữ liệu**
 - DML - *Data Manipulation Language*
 - Các lệnh dùng để bảo trì và truy vấn CSDL: thêm (insert), sửa (update), xóa (delete) dữ liệu của bảng, truy vấn (select).

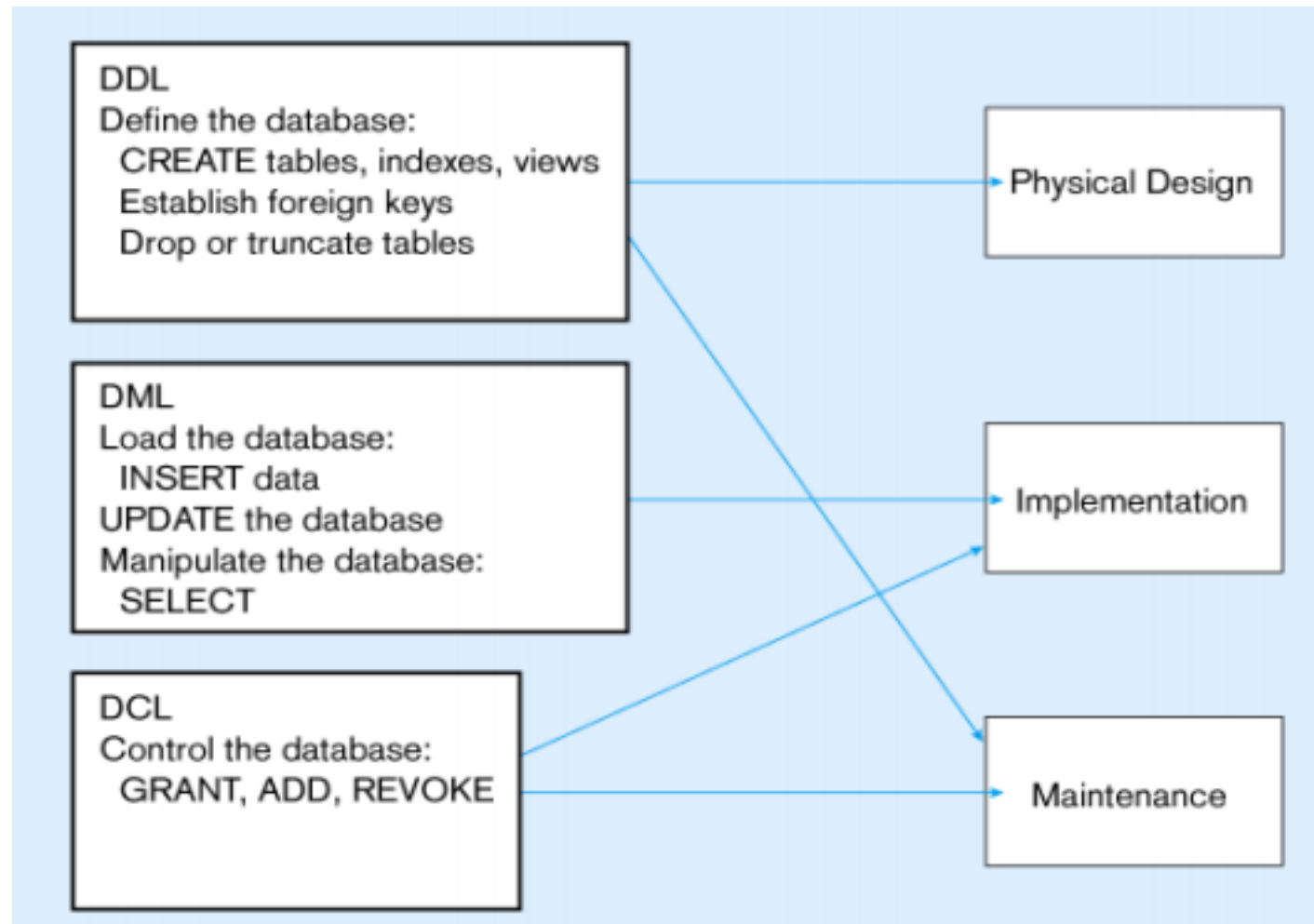


Ngôn ngữ SQL

- **Ngôn ngữ điều khiển dữ liệu**
 - DCL - *Data Control Language*
 - Các lệnh dùng để điều khiển CSDL: quản trị các quyền (grant, revoke) và ghi nhận (committing) dữ liệu.



Quá trình phát triển cơ sở dữ liệu



DDL, DML, DCL và quá trình phát triển CSDL.



Định nghĩa cơ sở dữ liệu

- **Các bước tạo một bảng**
 - **Bước 1.** Xác định kiểu dữ liệu của các cột.
 - **Bước 2.** Xác định các cột có thể hoặc không thể có giá trị rỗng (null value).
 - **Bước 3.** Xác định các cột phải có các giá trị duy nhất (các khóa dự tuyển).
 - **Bước 4.** Xác định khóa chính – khóa ngoại.
 - **Bước 5.** Xác định các giá trị mặc nhiên.
 - **Bước 6.** Xác định các ràng buộc trên các cột (mô tả miền trị).
 - **Bước 7.** Tạo bảng và các chỉ mục của bảng.



Định nghĩa cơ sở dữ liệu

```
CREATE TABLE tablename
( {column definition [table constraint] } . . .
[ON COMMIT {DELETE | PRESERVE} ROWS] );

where column definition ::=
column_name
    {domain name | datatype [(size)] }
    [column_constraint_clause . . .]
    [default value]
    [collate clause]

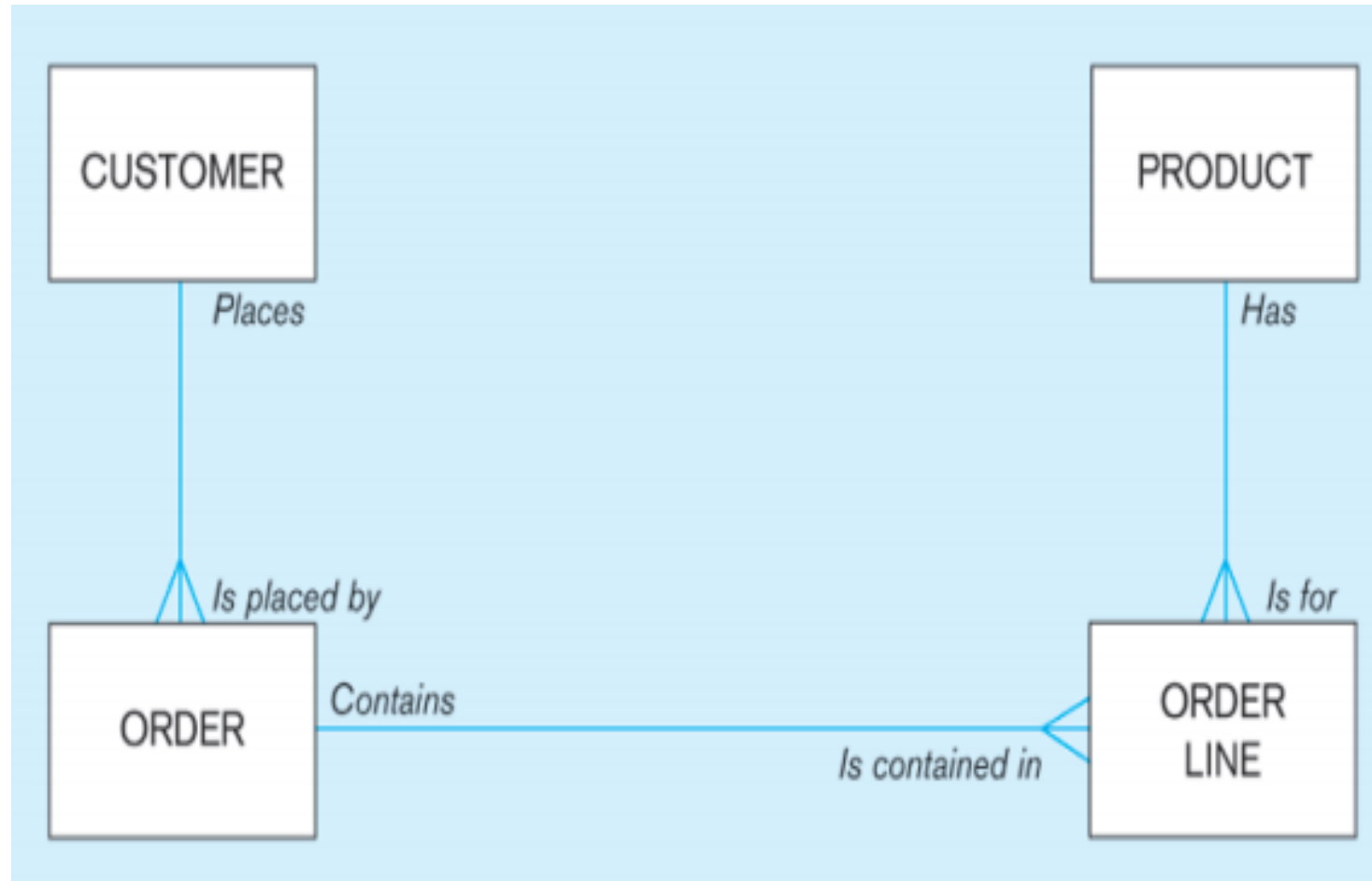
and table constraint ::=
    [CONSTRAINT constraint_name]
    Constraint_type [constraint_attributes]
```

```
CREATE TABLE <table name> [<list of columns>]
AS SELECT statement;
```

Cú pháp của lệnh CREATE TABLE.



Định nghĩa cơ sở dữ liệu



Định nghĩa cơ sở dữ liệu

```
CREATE TABLE CUSTOMER_T
(CUSTOMER_ID          NUMBER(11, 0) NOT NULL,
CUSTOMER_NAME         VARCHAR2(25) NOT NULL,
CUSTOMER_ADDRESS      VARCHAR2(30),
CITY                  VARCHAR2(20),
STATE                 VARCHAR2(2),
POSTAL_CODE           VARCHAR2(9),
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID));

CREATE TABLE ORDER_T
(ORDER_ID             NUMBER(11, 0) NOT NULL,
ORDER_DATE            DATE          DEFAULT SYSDATE,
CUSTOMER_ID           NUMBER(11, 0),
CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));

CREATE TABLE PRODUCT_T
(PRODUCT_ID           INTEGER      NOT NULL,
PRODUCT_DESCRIPTION   VARCHAR2(50),
PRODUCT_FINISH        VARCHAR2(20)
                     CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                                                'Red Oak', 'Natural Oak', 'Walnut')),
STANDARD_PRICE        DECIMAL(6,2),
PRODUCT_LINE_ID       INTEGER,
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));

CREATE TABLE ORDER_LINE_T
(ORDER_ID             NUMBER(11,0) NOT NULL,
PRODUCT_ID            NUMBER(11,0) NOT NULL,
ORDERED_QUANTITY      NUMBER(11,0),
CONSTRAINT ORDER_LINE_PK PRIMARY KEY (ORDER_ID, PRODUCT_ID),
CONSTRAINT ORDER_LINE_FK1 FOREIGN KEY(ORDER_ID) REFERENCES ORDER_T(ORDER_ID),
CONSTRAINT ORDER_LINE_FK2 FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT_T(PRODUCT_ID));
```



Tính toàn vẹn dữ liệu

- **Tính toàn vẹn tham chiếu** (*referential integrity*) là ràng buộc để bảo đảm các giá trị của khóa ngoại phải tương xứng với các giá trị của khóa chính của bảng liên quan trong mỗi liên kết một – nhiều.
- **Hạn chế**
 - Xóa bỏ các hàng của bảng có khóa chính.
 - Cập nhật các hàng của bảng có khóa chính.
 - Thêm các hàng vào bảng có khóa ngoại.

Tính toàn vẹn dữ liệu



Relational integrity is enforced via the primary-key to foreign-key match

Restricted Update: A customer ID can only be deleted if it is not found in ORDER table.

```
CREATE TABLE CUSTOMER_T  
    (CUSTOMER_ID      INTEGER DEFAULT 'C999' NOT NULL,  
     CUSTOMER_NAME    VARCHAR(40)          NOT NULL,  
     ...
```

```
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID),  
ON UPDATE RESTRICT);
```

Cascaded Update: Changing a customer ID in the CUSTOMER table will result in that value changing in the ORDER table to match.

```
... ON UPDATE CASCADE);
```

Set Null Update: When a customer ID is changed, any customer ID in the ORDER table that matches the old customer ID is set to NULL.

```
... ON UPDATE SET NULL);
```

Set Default Update: When a customer ID is changed, any customer ID in the ORDER tables that matches the old customer ID is set to a predefined default value.

```
... ON UPDATE SET DEFAULT);
```

Bảo đảm tính toàn vẹn dữ liệu khi cập nhật.



Thay đổi định nghĩa bảng

- Lệnh ***ALTER TABLE*** dùng để thay đổi định nghĩa của một bảng.

Cú pháp của lệnh **ALTER TABLE**.

ALTER TABLE <table name>

[ADD | MODIFY | DROP options]

(<column definition> [<column constraint>])

[ENABLE clause | DISABLE clause];



Thay đổi định nghĩa bảng

- ***Các tùy chọn của lệnh ALTER TABLE***
 - **ADD:** Thêm một cột và/hoặc các ràng buộc vào một bảng.
 - **MODIFY:** Thay đổi định nghĩa của một cột.
 - **DROP:** Hủy bỏ một ràng buộc của một bảng.



Thay đổi định nghĩa bảng

- ***Hạn chế của lệnh ALTER TABLE***
 - Không thể thay đổi một cột đang chứa các giá trị NULL thành NOT NULL.
 - Không thể thêm một cột mới với ràng buộc NOT NULL.
 - Không thể giảm kích thước hoặc thay đổi kiểu dữ liệu của một cột, trừ khi cột này không có chứa dữ liệu.
 - Không thể sử dụng tùy chọn MODIFY để định nghĩa các ràng buộc trên một cột ngoại trừ NULL/NOT NULL.



Thay đổi định nghĩa bảng

Thêm một cột và/hoặc các ràng buộc vào một bảng:

ALTER TABLE <table name>

ADD (<column definition> [<column constraint>]);

Thêm cột Type vào bảng Customer_T

ALTER TABLE Customer_T

ADD (Type VARCHAR(2));

Thêm ràng buộc của cột Standard_Price của bảng Product_T

ALTER TABLE Product_T

ADD (CHECK(Standard_Price > 0));



Thay đổi định nghĩa bảng

Thay đổi định nghĩa của một cột:

ALTER TABLE <table name>

MODIFY (<column name> <type> [NULL]);

**Thay đổi chiều dài của cột Customer_Name của bảng
Customer_T**

ALTER TABLE Customer_T

MODIFY (Customer_Name VARCHAR2(30));



Thay đổi định nghĩa bảng

Hủy bỏ ràng buộc của một bảng:

ALTER TABLE <table name>

DROP

[CONSTRAINT <constraint name>

| PRIMARY KEY

| UNIQUE (<list of columns>)]

[CASCADE];

Hủy bỏ ràng buộc khóa chính của bảng Order_line_T

ALTER TABLE Order_Line_T

DROP CONSTRAINT Order_Line_PK;

- **Tùy chọn CASCADE hủy bỏ tất cả ràng buộc khóa ngoại tham chiếu đến bảng Customer_T**

ALTER TABLE Customer_T

DROP CONSTRAINT Customer_PK CASCADE;



Thay đổi định nghĩa bảng

Cho phép một ràng buộc có hiệu lực / không có hiệu lực:

```
ALTER TABLE <table name>  
[ENABLE | DISABLE]  
[CONSTRAINT <constraint name>  
|PRIMARY KEY  
|UNIQUE (<list of columns>)]  
[CASCADE];
```

**Làm mất hiệu lực của ràng buộc Customer_PK của
bảng Customer_T**

```
ALTER TABLE Customer_T  
DISABLE CONSTRAINT Customer_PK CASCADE;
```



Hủy bỏ bảng

- Lệnh ***DROP TABLE*** dùng để hủy bỏ một bảng trong một lược đồ.

Cú pháp của lệnh **DROP TABLE**:

DROP TABLE <table name> [CASCADE CONSTRAINTS];

Hủy bỏ bảng Order_Line_T

DROP TABLE Order_Line_T;



Định nghĩa lược đồ

- **Tạo chỉ mục**

- Truy xuất nhanh (ngẫu nhiên / tuần tự) dữ liệu của bảng cơ sở.

Cú pháp của lệnh **CREATE INDEX**:

**CREATE [UNIQUE] INDEX <index name>
ON <table name> (<list of columns>);**

Tạo chỉ mục Name_IDX trên Customer_Name

**CREATE INDEX Name_IDX
ON Customer_T (Customer_Name);**



Định nghĩa lược đồ

- *Hủy bỏ chỉ mục*

Cú pháp của lệnh **DROP INDEX**:

DROP INDEX <index name>;

Hủy bỏ chỉ mục Name_IDX

DROP INDEX Name_IDX;



Lệnh *INSERT*

- ***Thêm dữ liệu vào một bảng***

Cú pháp của lệnh *INSERT* - Thêm một hàng:

**INSERT INTO <table name> [(<list of columns>)]
VALUES (<list of expressions>);**

Cú pháp của lệnh *INSERT* - Thêm nhiều hàng:

**INSERT INTO <table name> [(<list of columns>)]
SELECT statement;**



Lệnh *INSERT*

```
INSERT INTO Customer_T  
VALUES (001, 'Contemporary Casuals',  
'1355 S. Himes Blvd.', 'Gainesville', 'FL', 32601);
```

```
INSERT INTO Product_T (Product_ID,  
Product_Description, Product_Finish,  
Standard_Price, Product_On_Hand)  
VALUES (1, 'End Table', 'Cherry', 175, 8);
```

```
INSERT INTO CA_Customer_T  
SELECT *  
FROM Customer_T  
WHERE State = 'CA';
```



Lệnh *DELETE*

- ***Xóa bỏ các hàng của một bảng***

Cú pháp của lệnh **DELETE**:

```
DELETE [FROM] <table name>  
[WHERE <row conditions>];
```

Xóa một số hàng của bảng Customer_T

```
DELETE FROM Customer_T  
WHERE State = 'HI';
```

Xóa tất cả các hàng của bảng Customer_T

```
DELETE FROM Customer_T ;
```



Lệnh *UPDATE*

- **Cập nhật dữ liệu của các hàng của một bảng**

Cú pháp của lệnh **UPDATE**:

UPDATE <table name> [<alias>]

SET <column1> = {<expression>, <subquery>}

[, <column2> = {<expression>, <subquery>} ...]

[**WHERE** <row conditions>];

Cập nhật một số hàng của bảng Product_T

UPDATE Product_T

SET Unit_Price = 775

WHERE Product_ID = 7;



Lệnh *SELECT*

- Dùng để truy vấn dữ liệu của một bảng hoặc nhiều bảng.
- Lệnh ***SELECT*** thực hiện các phép toán của đại số quan hệ.
 - Phép tích
 - Phép kết
 - Phép chọn
 - Phép chiếu



Lệnh *SELECT*

- Dùng để truy vấn dữ liệu của một bảng hoặc nhiều bảng.
- Lệnh ***SELECT*** thực hiện các phép toán của đại số quan hệ.
 - Phép tích
 - Phép kết
 - Phép chọn
 - Phép chiếu



Lệnh *SELECT*

Cú pháp của lệnh *SELECT*:

```
SELECT [DISTINCT] <list of expressions>  
[INTO <list of variables>  
FROM <list of tables>  
[WHERE <row conditions>  
[GROUP BY <list of expressions>  
[HAVING <group conditions>]]  
[ORDER BY <list of expressions>];
```

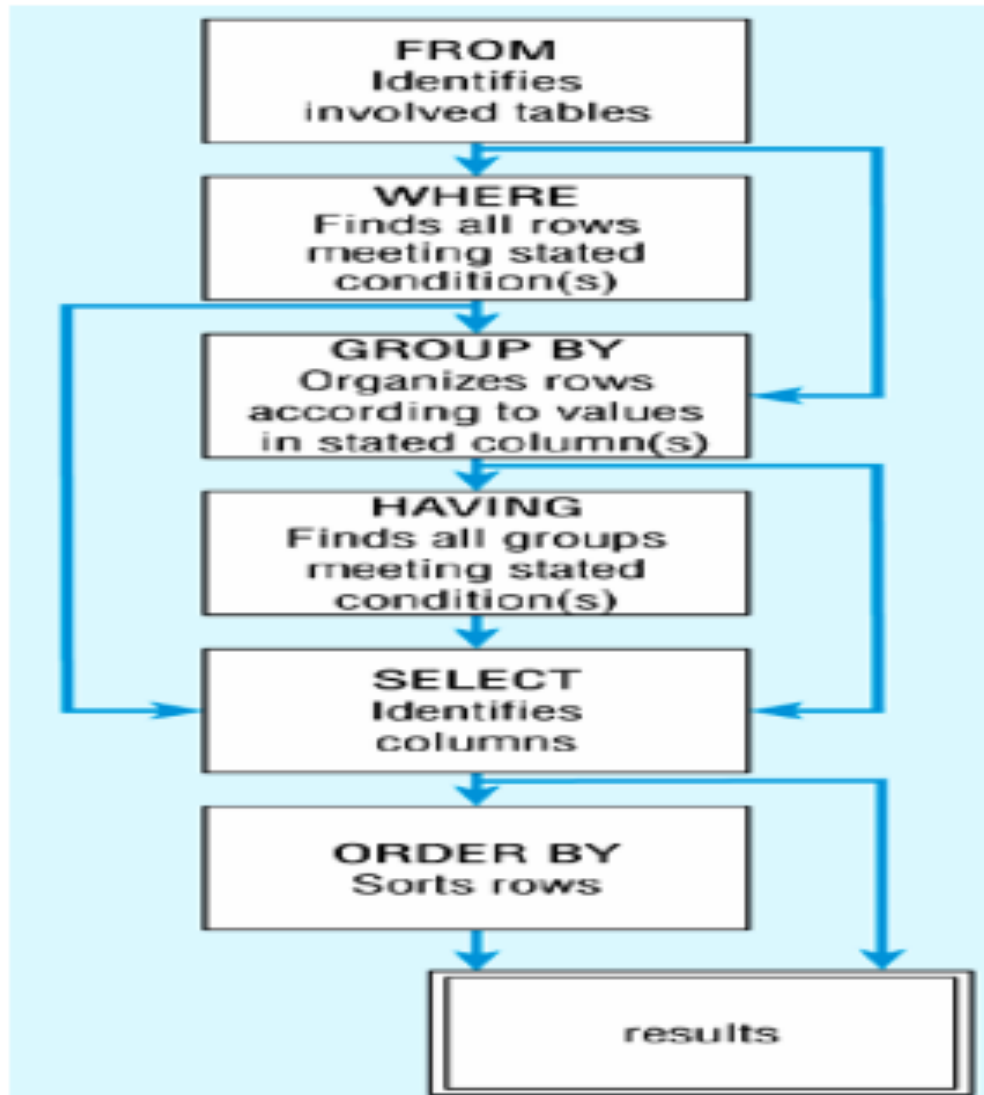


Lệnh *SELECT*

- **Các mệnh đề của lệnh *SELECT***

- **SELECT:** liệt kê các cột (các biểu thức) của kết quả.
- **FROM:** các bảng hoặc các khung nhìn chứa dữ liệu cần thiết cho truy vấn.
- **WHERE:** điều kiện xử lý các hàng để tạo ra kết quả.
- **GROUP BY:** gom nhóm các hàng.
- **HAVING:** điều kiện xử lý các nhóm.
- **ORDER BY:** sắp thứ tự kết quả.

Lệnh *SELECT*



Thứ tự xử lý các mệnh đề của lệnh *SELECT*.



Lệnh *SELECT*

- **Mệnh đề *SELECT***

- ***SELECT* [DISTINCT] <list of expressions>**
- Thực hiện ***phép chiếu*** của đại số quan hệ.
- ***list of expressions*** là danh sách các biểu thức dùng để tạo ra kết quả, các biểu thức này được phân cách nhau bởi dấu phẩy.
- Bí danh cột (*column alias*).
- Dấu *.
- Cột có thể có dạng:
table_name.column_name
- Từ khóa **DISTINCT**.



Lệnh *SELECT*

- Mệnh đề *INTO*

- *INTO* <*list of variables*>
- Được sử dụng trong Oracle PL/SQL, dùng để gán giá trị của các biểu thức cho các biến theo thứ tự tương ứng.
- *List of variables* là danh sách các biến được phân cách nhau bởi dấu phẩy.



Lệnh *SELECT*

- **Mệnh đề FROM**

- **FROM** *<list of tables>*
- Thực hiện **phép tích** của đại số quan hệ, dùng để chỉ ra các bảng chứa dữ liệu cần lấy ra.
- **List of tables** là danh sách các bảng được phân cách nhau bởi dấu phẩy.
- Bí danh bảng (*table alias*).



Lệnh *SELECT*

SELECT *

FROM Order_T;

SELECT Order_ID, Order_Date, Customer_ID
FROM Order_T;

SELECT DISTINCT Order_Date “Date of Order”
FROM Order_T;

SELECT Order_ID **AS Identifier**, Order_Date **Date**
FROM Order_T;



Lệnh *SELECT*

- Mệnh đề *WHERE*
 - *WHERE* <row conditions>
 - Thực hiện các *phép chọn, phép kết* của đại số quan hệ.
 - *Row conditions* là các điều kiện được xét trên mỗi hàng, các hàng nào thỏa mãn các điều kiện này thì được đưa vào kết quả của truy vấn.
 - Để lệnh *SELECT* có ngữ nghĩa, nếu mệnh đề *FROM* có n bảng thì mệnh đề *WHERE* phải có n-1 điều kiện kết.



Lệnh *SELECT*

Các phép toán so sánh

=	Bằng
<	Nhỏ hơn
<=	Nhỏ hơn hoặc bằng
>	Lớn hơn
>=	Lớn hơn hoặc bằng
<>	Không bằng
!=	Không bằng
BETWEEN ... AND ...	Giữa hai giá trị
IN (list)	Một trong các giá trị của danh sách
LIKE string	Giống khuôn mẫu
IS NULL	Là giá trị rỗng

Các phép toán phủ định

NOT BETWEEN ... AND ...
NOT IN (list)
NOT LIKE string
IS NOT NULL



Lệnh *SELECT*

NOT	True	False	Null
	False	True	Null

AND	True	False	Null
True	True	False	Null
False	False	False	False
Null	Null	False	Null

OR	True	False	Null
True	True	True	True
False	True	False	Null
Null	True	Null	Null



Lệnh *SELECT*

```
SELECT Product_ID, Standard_Price  
FROM Product_T  
WHERE Standard_Price < 275;
```

```
SELECT Cust.Customer_Name AS Name,  
Customer_Address  
FROM Customer_T Cust  
WHERE Customer_Name = 'Home Furnishings';
```

```
SELECT Product_ID, Standard_Price  
FROM Product_  
WHERE Standard_Price BETWEEN 100 AND 200;
```

```
SELECT Customer_Name, City, State  
FROM Customer_T  
WHERE Customer_State IN ('FL', 'TX', 'CA', 'HI');
```



Lệnh *SELECT*

```
SELECT Product_Description, Product_Finish,  
Standard_Price  
FROM Product_T  
WHERE (Product_Description LIKE '%Desk'  
OR Product_Description LIKE '_A%')  
AND Standard_Price > 300;
```

```
SELECT Product_ID, Product_Finish, Standard_Price  
FROM Product_T  
WHERE Product_Description IS NULL;
```

```
SELECT COUNT(*)  
FROM Order_Line_T  
WHERE Order_ID = 1004;
```



Lệnh *SELECT*

- Mệnh đề ***GROUP BY***
 - ***GROUP BY <list of expressions>***
 - Dùng để phân chia các hàng của một bảng thành các nhóm nhỏ hơn.
 - Các hàm nhóm có thể được sử dụng để trả về thông tin chung cho mỗi nhóm.
 - Mỗi nhóm chỉ xuất hiện ở một hàng trong kết quả của truy vấn.
 - Thứ tự thực hiện: mệnh đề ***WHERE***, mệnh đề ***GROUP BY***.



Lệnh *SELECT*

- Mệnh đề ***GROUP BY***

- Các cột trong mệnh đề ***GROUP BY*** không bắt buộc phải có trong mệnh đề ***SELECT***.
- Khi có mệnh đề ***GROUP BY***, tất cả các cột có trong mệnh đề ***SELECT*** phải có trong mệnh đề ***GROUP BY***, ngoại trừ chúng ở trong hàm nhóm.
- Nếu lệnh ***SELECT*** không có mệnh đề ***GROUP BY*** thì toàn bộ bảng được xem là một nhóm.
- Nếu mệnh đề ***SELECT*** có chứa hàm nhóm thì không thể lấy được chi tiết của mỗi hàng của nhóm.



Lệnh *SELECT*

```
SELECT State, COUNT(State)
FROM Customer_T
GROUP BY State;
```

```
SELECT State, COUNT(State)
FROM Customer_T
WHERE State IN ('FL', 'TX', 'CA', 'HI')
GROUP BY State;
```



Lệnh *SELECT*

- Mệnh đề ***HAVING***
 - ***HAVING*** <group conditions>
 - Dùng để xác định các nhóm được đưa vào kết quả của truy vấn.
 - ***Group conditions*** là các điều kiện được xét cho mỗi nhóm.
 - Tất cả các cột có trong mệnh đề ***HAVING*** phải có trong mệnh đề ***GROUP BY***, ngoại trừ chúng ở trong hàm nhóm.



Lệnh *SELECT*

- Mệnh đề *HAVING*

- Các nhóm được tạo ra và các hàm nhóm được tính toán trước khi thực hiện mệnh đề *HAVING* để chọn ra các nhóm.
- Mệnh đề *WHERE* không được dùng để chọn các nhóm.
- Thứ tự thực hiện: mệnh đề *WHERE*, mệnh đề *GROUP BY*, mệnh đề *HAVING*.



Lệnh *SELECT*

```
SELECT State, COUNT(State)
FROM Customer_T
GROUP BY State
HAVING COUNT(State) > 1;
```

```
SELECT State, COUNT(State)
FROM Customer_T
WHERE State IN ('FL', 'TX', 'CA', 'HI')
GROUP BY State
HAVING COUNT(State) > 1;
```



Lệnh *SELECT*

- Mệnh đề ***ORDER BY***
 - ***ORDER BY <list of expressions>***
 - Mệnh đề ***ORDER BY*** được dùng để sắp thứ tự các hàng được trả về trong kết quả của truy vấn.
 - Mệnh đề ***ORDER BY*** luôn luôn là mệnh đề cuối cùng của lệnh ***SELECT***.
 - Thứ tự ngầm định là tăng dần (***ASC*** – ***ASCending***), từ khóa ***DESC*** (***DEScending***) đứng ngay sau tên cột dùng để chỉ định thứ tự giảm dần.
 - Các cột trong mệnh đề ***ORDER BY*** không bắt buộc phải có trong mệnh đề ***SELECT***.



Lệnh *SELECT*

```
SELECT State, COUNT(State)
FROM Customer_T
WHERE State IN ('FL', 'TX', 'CA', 'HI')
GROUP BY State
HAVING COUNT(State) > 1
ORDER BY State DESC;
```



Hàm kết hợp

- **Hàm kết hợp** (*aggregate function*) còn được gọi là hàm nhóm (*group function*).

Hàm	Giá trị trả về
AVG ([DISTINCT <u>ALL</u>] n)	Giá trị trung bình của <i>n</i> , bỏ qua các giá trị rỗng.
COUNT ([DISTINCT <u>ALL</u>] expr *)	Số hàng mà <i>expr</i> có giá trị khác rỗng. * làm cho COUNT đếm tất cả các hàng được chọn, bao gồm các hàng trùng nhau và các hàng có giá trị rỗng.
MAX ([DISTINCT <u>ALL</u>] expr)	Giá trị lớn nhất của <i>expr</i> .
MIN ([DISTINCT <u>ALL</u>] expr)	Giá trị nhỏ nhất của <i>expr</i> .
SUM ([DISTINCT <u>ALL</u>] n)	Tổng giá trị của <i>n</i> , bỏ qua các giá trị rỗng.
STDDEV ([DISTINCT <u>ALL</u>] n)	Độ lệch chuẩn (<i>STanDard DEViation</i>) của <i>n</i> , bỏ qua các giá trị rỗng.
VARIANCE ([DISTINCT <u>ALL</u>] n)	Phương sai của <i>n</i> , bỏ qua các giá trị rỗng.



Hàm kết hợp

- **Sử dụng các hàm kết hợp**

- **Kết hợp đơn** (scalar aggregate): truy vấn trả về một giá trị của hàm kết hợp.
- **Kết hợp vector** (vector aggregate): truy vấn trả về nhiều giá trị của hàm kết hợp (dùng **GROUP BY**).

```
SELECT MAX(Standard_Price), MIN(Standard_Price)  
FROM Product_T
```

```
SELECT Customer_State, COUNT(Customer_State)  
FROM Customer_T  
GROUP BY Customer_State
```



Thứ tự thực hiện ưu tiên của phép toán

- Các phép toán có cùng độ ưu tiên sẽ được thực hiện từ trái qua phải:
 - Biểu thức con trong dấu ngoặc
 - Các phép toán số học $*$, $/$
 - Các phép toán số học $+$, $-$
 - Các phép toán so sánh và các phép toán
 - SQL: $=$, $\neq$, $<$, $\leq$, $>$, $\geq$, BETWEEN ... AND, IN, LIKE, IS NULL
 - NOT
 - AND
 - OR



Các phép toán tập hợp

- Hai lệnh **SELECT** có thể được kết nối với nhau bằng các phép toán tập hợp bao gồm phép hợp (union), phép giao (intersect) và phép hiệu (except/minus).

```
SELECT statement_1  
UNION [ALL] | INTERSECT | EXCEPT  
SELECT statement_2
```



Các phép toán tập hợp

- ***UNION*** thực hiện phép hợp kết quả của hai truy vấn.
- ***INTERSECT*** thực hiện phép giao kết quả của hai truy vấn.
- ***EXCEPT/MINUS*** thực hiện phép hiệu kết quả của hai truy vấn.
- Từ khóa ***ALL*** cho phép các hàng trong kết quả có thể trùng nhau.



Các phép toán tập hợp

```
SELECT Customer_State  
FROM Customer_T  
WHERE State IN ('FL', 'TX', 'CA', 'HI')
```

```
SELECT Customer_State  
FROM Customer_T  
EXCEPT  
SELECT Customer_State  
FROM Customer_T  
WHERE Customer_State IN ('FL', 'TX', 'CA', 'HI')
```



Xử lý giá trị rỗng

- Hàm **ISNULL** (*Null VaLue*) dùng để đổi giá trị rỗng thành một giá trị khác rỗng.
- Hàm **ISNULL** có hai tham số: một biểu thức và một giá trị khác rỗng.
- Nếu giá trị của các cột trong một biểu thức là null thì giá trị của biểu thức này là null.

```
SELECT Product_ID, ISNULL(Standard_Price, 0)  
FROM Product_T  
WHERE Standard_Price IS NULL
```



Định nghĩa và sử dụng khung nhìn

- **Quan hệ cơ sở**

- **Quan hệ cơ sở (*base relation*)** là một quan hệ được đặt tên, tương ứng với một kiểu thực thể trong lược đồ ý niệm, mà các bộ được lưu trữ vật lý trong cơ sở dữ liệu.

- **Bảng cơ sở**

- **Bảng cơ sở (*base table*)** là bảng chứa dữ liệu thô (raw data).

- **Khung nhìn**

- **Khung nhìn (*view*)** là kết quả của một hoặc nhiều phép toán quan hệ thực hiện trên các quan hệ cơ sở để tạo ra một quan hệ khác.



Định nghĩa và sử dụng khung nhìn

- **Khung nhìn là một quan hệ ảo**
 - Không tồn tại thật sự trong CSDL.
 - Được tạo ra khi người sử dụng tham chiếu đến nó.
 - Cho phép người sử dụng truy xuất các bảng.
 - Mỗi khung nhìn có một tên.
 - Khung nhìn được tạo từ lệnh *SELECT*.
 - *CHECK_OPTION* chỉ dùng cho các khung nhìn có thể cập nhật được (updatable view) và ngăn cấm việc thêm các hàng mới vào khung nhìn nếu các hàng này không thỏa mãn điều kiện *WHERE* của truy vấn định nghĩa khung nhìn này.



Định nghĩa và sử dụng khung nhìn

- **Khung nhìn động**
 - **Khung nhìn động** (*dynamic view*) là một bảng ảo được tạo ra khi người sử dụng tham chiếu đến.
 - Không tồn tại thực sự trong CSDL; dữ liệu của khung nhìn được lấy từ các bảng cơ sở và các khung nhìn khác.
 - Được tạo từ lệnh *SELECT* dựa trên các bảng cơ sở và các khung nhìn khác.



Định nghĩa và sử dụng khung nhìn

- **Khung nhìn hiện thực**
 - **Khung nhìn hiện thực** (*materialized view*) là một bản sao (*copy*) hoặc bản nhân (*replica*) của dữ liệu.
 - Tồn tại thực sự như là một bảng trong CSDL.
 - Được đồng bộ hóa với các bảng cơ sở tạo ra nó.



Định nghĩa và sử dụng khung nhìn

```
CREATE VIEW Expensive_Stuff_V  
AS  
SELECT Product_ID, Product_Name, Unit_Price  
FROM Product_T  
WHERE Standard_Price > 300  
WITH CHECK OPTION
```



Định nghĩa và sử dụng khung nhìn

- **Mục đích của khung nhìn**

- Hạn chế truy xuất vào CSDL, hiển thị một phần của CSDL, cho phép bảo mật dữ liệu (*data security*) vì không nhìn thấy các bảng cơ sở tạo ra khung nhìn.
- Cho phép người sử dụng thực hiện các truy vấn đơn giản để lấy kết quả từ các truy vấn phức tạp, làm tăng hiệu suất lập trình (*programming productivity*).
- Cung cấp tính độc lập dữ liệu cho người sử dụng, cho phép những người sử dụng khác nhau nhìn thấy cùng dữ liệu theo các cách khác nhau.



Định nghĩa và sử dụng khung nhìn

- **Ưu điểm của khung nhìn**

- Tính độc lập dữ liệu
- Nâng cao tính bảo mật
- Nâng cao hiệu suất lập trình
- Tính thông hành (currency)
- Tùy biến (customization)
- Toàn vẹn dữ liệu
- Tiện lợi

- **Nhược điểm của khung nhìn**

- Hạn chế cập nhật
- Hạn chế cấu trúc
- Giảm hiệu suất thực hiện



Xử lý nhiều bảng - Phép kết

- **Phép kết**

- **Phép kết** (*join*) là một phép toán quan hệ (relational operation) cho phép kết hai hoặc nhiều bảng dựa vào một miền chung để tạo thành một bảng hoặc khung nhìn.

- **Phép kết bằng**

- **Phép kết bằng** (*equi-join*) là phép kết mà điều kiện kết là các giá trị cột chung; các cột chung vẫn xuất hiện dư thừa trong bảng kết quả.



Xử lý nhiều bảng - Phép kết

- **Phép kết tự nhiên**

- **Phép kết tự nhiên** (*natural join*) là phép kết bằng nhưng các cột chung chỉ xuất hiện duy nhất trong bảng kết quả.

- **Phép kết ngoài**

- **Phép kết ngoài** (*outer join*) là phép kết mà các hàng không có các giá trị tương ứng trong các cột chung cũng được đưa vào bảng kết quả.
- Thông thường, các cột chung của các bảng kết với nhau là khóa chính của bảng chính (*dominant table*) và khóa ngoại của bảng phụ thuộc (*dependent table*) trong các mối liên kết một-nhiều.

Phép kết ngoài

```
SELECT Customer_T.Customer_ID, Customer_Name, Order_ID  
FROM Customer_T LEFT OUTER JOIN Order_T  
ON Customer_T.Customer_ID = Order_T.Customer_ID;
```

Cú pháp Microsoft

LEFT OUTER JOIN syntax with ON keyword instead of WHERE →
causes customer data to appear even if there is no corresponding
order data

```
SELECT Customer_T.Customer_ID, Customer_Name, Order_ID  
FROM Customer_T, Order_T  
WHERE Customer_T.Customer_ID = Order_T.Customer_ID(+);
```

Cú pháp Oracle

Outer join in Oracle uses regular join syntax, but
adds (+) symbol to the side that will have the
missing data



Truy vấn con

- **Truy vấn con**

- Truy vấn con (*subquery*) là một truy vấn (lệnh *SELECT*) nằm trong một truy vấn khác.
- Truy vấn ngoài (*outer query*)
- Truy vấn trong (*inner query*)
- Truy vấn chính (*main query*)

- **Xuất hiện**

- Trong điều kiện của mệnh đề *WHERE*.
- Như là một bảng trong mệnh đề *FROM*.
- Trong điều kiện của mệnh đề *HAVING*.



Truy vấn con

- **Các loại truy vấn con**

- **Truy vấn con lồng nhau (nested subquery)**

- Không phụ thuộc vào dữ liệu của truy vấn ngoài.
- Được thực hiện duy nhất một lần trước khi thực hiện truy vấn ngoài.
- Kết quả của truy vấn con được dùng để thực hiện truy vấn ngoài.

- **Truy vấn con tương quan (correlated subquery)**

- Sử dụng dữ liệu của truy vấn ngoài.
- Đối với mỗi hàng của truy vấn ngoài, dữ liệu của hàng này được dùng để thực hiện truy vấn con, kết quả của truy vấn con được dùng để thực hiện truy vấn ngoài.
- Có thể sử dụng phép toán *EXISTS*.

Truy vấn con

```
SELECT Customer_Name  
FROM Customer_T  
WHERE Customer_ID IN
```

The IN operator will test to see if the Customer_ID value of a row is included in the list returned from the subquery

```
(SELECT DISTINCT Customer_ID  
FROM Order_T);
```

Subquery is embedded in parentheses. In this case it returns a list that will be used in the WHERE clause of the outer query

Ví dụ truy vấn con lồng nhau

Truy vấn con

```
SELECT DISTINCT Order_ID  
FROM Order_Line_T  
WHERE EXISTS
```

```
(SELECT *  
FROM Product_T  
WHERE Product_ID = Order_Line_T.Product_ID  
AND Product_Finish = 'Natural ash');
```

The EXISTS operator will return a TRUE value if the subquery resulted in a non-empty set, otherwise it returns a FALSE

The subquery is testing for a value that comes from the outer query

Ví dụ truy vấn con tương quan



Truy vấn con

Subquery forms the derived table used in the **FROM** clause of the outer query

One column of the subquery is an aggregate function that has an alias name. That alias can then be referred to in the outer query

```
SELECT Product Description, Standard Price, Avg_Price  
FROM (SELECT AVG(Standard_Price) Avg_Price  
      FROM Product_T), Product_T  
WHERE Standard_Price > Avg_Price;
```

The **WHERE** clause normally cannot include aggregate functions, but because the aggregate is performed in the subquery its result can be used in the outer query's **WHERE** clause

Ví dụ truy vấn con xuất hiện trong mệnh đề **FROM**



Tài liệu tham khảo

- ✓ Sách “**Hệ quản trị Cơ sở dữ liệu**” của Nhà xuất bản ĐH Quốc Gia TP Hồ Chí Minh (Tác giả : Dương Anh Tuấn, Nguyễn Trung Trực).
- ✓ Bài giảng “**Ngôn ngữ SQL**” của Thầy Nguyễn Trung Trực – Khoa Công Nghệ Thông Tin, Trường ĐH Quốc Gia TP Hồ Chí Minh).



VANLANG UNIVERSITY
School of Technology
Information Technology

